Studio Nx

JS 298 — 내보내기 계획 계산

2026-10-20

JavaScript 298 — 내보내기 계획 계산 (`audio/export_plan.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/export_plan.js`
작업	S1-11b WAV 출력: 창에서 고른 옵션을 "어디서부터 어디까지, 꼬리는 몇 초, 파일명은 무엇"으로 바꿉니다 (FR-301, FR-307)
작성	Claude (AI), 2026-10-20, 사용자 검토
이전, 다음	이전 297 WAV 인코딩, 다음 299 오프라인 렌더링

1. 하는 일

내보내기 창의 옵션을 숫자로 바꾸는 순수 함수 모음입니다. 화면이나 소리 장치를 쓰지 않아서 Node에서 시험합니다.

함수	입력, 출력
`projectEndTick`	모든 클립 중 가장 뒤에 끝나는 tick (곡의 끝)
`resolveExportRange`	범위 모드(전체, 루프, 선택한 클립, 직접 지정)와 현재 상태를 받아 시작 tick과 끝 tick을 돌려줍니다. 만들 수 없으면 이유를 돌려줍니다.
`rangeSeconds`	범위의 길이(초). 템포 변화를 반영합니다.
`autoTailSeconds`	꼬리 "Auto" 값: 이펙트와 악기가 범위 끝 뒤로 남기는 소리의 길이
`sanitizeFileName`, `exportFileName`	파일명 (`{프로젝트명}_{YYYYMMDD_HHmm}`)
`formatDuration`, `formatBytes`	창에 보여 줄 `1:05`, `11.8 MB` 같은 글자

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-export.js`의 `refreshForm`과 `openExport` (300)	범위별 설명, 요약 줄, 기본 파일명, 꼬리 Auto 값을 채웁니다.
`sequencer-export.js`의 `startExport` (300)	렌더링에 넘길 시작 tick과 끝 tick을 구합니다.

3. 코드

3.1 곡의 끝과 범위

```
export function projectEndTick(clips) {
  let end = 0;
  (clips || []).forEach(function (c) {
    const e = c.start_tick + c.length_tick;
    if (e > end) end = e;
  });
  return end;
}
```

```
export function resolveExportRange(opt) {
  const tpb = opt.ticksPerBar;
  if (opt.mode === 'all') {
    const end = projectEndTick(opt.clips);
    return end > 0 ? { ok: true, startTick: 0, endTick: end } : { ok: false, reason: 'empty' };
  }
  if (opt.mode === 'loop') {
    const loop = opt.loop;
    if (!loop || !(loop.end > loop.start)) return { ok: false, reason: 'no-loop' };
    return { ok: true, startTick: loop.start, endTick: loop.end };
  }
  if (opt.mode === 'selected') {
    const sel = opt.selected || [];
    if (sel.length === 0) return { ok: false, reason: 'no-selection' };
    let start = Infinity;
    sel.forEach(function (c) { if (c.start_tick < start) start = c.start_tick; });
    const end = projectEndTick(sel);
    return end > start ? { ok: true, startTick: start, endTick: end } : { ok: false, reason: 'no-selecti
on' };
  }
  if (opt.mode === 'custom') {
    const from = Math.floor(Number(opt.fromBar));
    const to = Math.floor(Number(opt.toBar));
    if (!(from >= 1) || !(to >= from)) return { ok: false, reason: 'bad-range' };
    return { ok: true, startTick: (from - 1) * tpb, endTick: to * tpb };
  }
  return { ok: false, reason: 'bad-range' };
}
```

```
export function rangeSeconds(startTick, endTick, baseBpm, changes) {
  return ticksToSecondsWithChanges(endTick, baseBpm, changes) - ticksToSecondsWithChanges(startTick, bas
eBpm, changes);
}
```

3.2 꼬리 Auto

```
export function autoTailSeconds(effects, layers, bpm) {
  let tail = 0;
  (effects || []).forEach(function (e) {
    if (e.is_bypassed) return;
    const p = resolveParams(e.schema, e.params);
    if (e.plugin_code === 'reverb') {
      tail = Math.max(tail, (Number(p.decay) || 2) + (Number(p.pre_delay) || 0));
    } else if (e.plugin_code === 'delay') {
      const fb = Math.max(0, Math.min(0.95, Number(p.feedback) || 0));
      const repeats = fb > 0 ? Math.min(60, Math.ceil(Math.log(0.001) / Math.log(fb))) : 1;
      tail = Math.max(tail, delaySeconds(p, bpm) * (repeats + 1));
    }
  });
  (layers || []).forEach(function (l) {
    if (l.is_enabled === false) return;
    const p = resolveParams(l.schema, l.params);
    if (p.env_release != null) tail = Math.max(tail, Number(p.env_release) || 0);
  });
```

```
  return Math.min(MAX_TAIL_SECONDS, Math.ceil(tail * 2) / 2);
}
```

3.3 파일명과 표시

```
export function sanitizeFileName(name) {
  const cleaned = String(name == null ? '' : name).replace(/[\\/:*?"<>|\u0000-\u001f]/g, '_').replace(/
\s+/g, ' ').trim();
  return cleaned.replace(/^\.+/, '').slice(0, 80) || 'Studio Nx';
}
```

```
export function exportFileName(title, date) {
  const two = function (n) { return (n < 10 ? '0' : '') + n; };
  const stamp = date.getFullYear() + two(date.getMonth() + 1) + two(date.getDate()) + '_' + two(date.get
Hours()) + two(date.getMinutes());
  return sanitizeFileName(title) + '_' + stamp;
}
```

```
export function formatDuration(seconds) {
  const s = Math.max(0, Math.round(seconds));
  const m = Math.floor(s / 60);
  const r = s % 60;
  return m + ':' + (r < 10 ? '0' : '') + r;
}
```

```
export function formatBytes(bytes) {
  if (bytes < 1024 * 1024) return Math.max(1, Math.round(bytes / 1024)) + ' KB';
  return (bytes / (1024 * 1024)).toFixed(1) + ' MB';
}
```

4. 범위 모드

모드	시작	끝	못 만드는 경우 (reason)
`all` 전체	0	가장 뒤 클립의 끝	클립이 하나도 없으면 `empty`
`loop` 루프 구간	루프 시작	루프 끝	루프가 꺼져 있으면 `no-loop`
`selected` 선택한 클립	선택한 클립 중 가장 앞 시작	선택한 클립 중 가장 뒤 끝	선택이 없으면 `no-selection`
`custom` 직접 지정	`fromBar` 마디의 처음	`toBar` 마디의 끝 (그 마디도 포함)	마디 번호가 1 미만이거나 끝이 시작보다 앞이거나 숫자가 아니면 `bad-range`

마디 번호는 화면과 같이 1부터 셉니다. 3마디부터 4마디까지는 `(3 - 1) × 3840`에서 `4 × 3840` tick입니다. 같은 마디(3에서 3)는 한 마디 길이가 됩니다.

5. 최신 문법으로 쓰면

```
// projectEndTick 과 sanitizeFileName 을 최신 문법으로 쓴 모양
export const projectEndTick = (clips = []) =>
  Math.max(0, ...clips.map((c) => c.start_tick + c.length_tick));
```

```
export const sanitizeFileName = (name) =>
  String(name ?? '').replace(/[\\/:*?"<>|\u0000-\u001f]/g, '_').replace(/\s+/g, ' ').trim()
    .replace(/^\.+/, '').slice(0, 80) || 'Studio Nx';

// exportFileName 의 두 자리 맞춤
const two = (n) => String(n).padStart(2, '0');
```

전통 문법 (실제 코드)	최신 문법
`forEach`로 최댓값 갱신	`Math.max(...배열)` (클립이 수만 개면 스프레드는 위험해서 실제 코드는 반복문)
`name == null ? '' : name`	`name ?? ''`
`(n < 10 ? '0' : '') + n`	`String(n).padStart(2, '0')`

6. 왜 이렇게

선택	이유
"전체"는 0에서 가장 뒤 클립의 끝까지입니다	FL Studio와 Ableton Live도 곡의 처음부터 마지막 클립 끝까지 내보냅니다. 마커나 눈금은 곡의 끝으로 보지 않습니다(화면 위의 마커가 35마디까지 있어도 클립이 6마디에서 끝나면 6마디까지 내보냅니다).
선택한 클립의 범위는 가장 앞 시작에서 가장 뒤 끝까지입니다	클립 사이에 빈 구간이 있어도 그 시간 관계를 유지하려는 것입니다. 선택하지 않은 클립의 소리는 `offline.js`에서 뺍니다.
직접 지정의 끝 마디는 포함합니다	사용자가 "9마디부터 25마디"라고 쓰면 25마디의 소리까지 듣고 싶다는 뜻이기 때문입니다.
못 만드는 경우를 예외가 아니라 `{ ok: false, reason }`으로 돌려줍니다	창이 열려 있는 동안 입력이 바뀔 때마다 부르는 함수입니다. 입력 중에는 잘못된 값이 흔하므로 오류로 끊지 않고, 이유를 보여 주고 내보내기 버튼만 끕니다.
꼬리 Auto는 효과별로 가장 긴 값을 고릅니다	리버브는 `decay + pre_delay`, 딜레이는 되먹임이 -60dB로 줄 때까지의 반복(`ln 0.001 / ln 되먹임`번)에 딜레이 시간을 곱한 값, 악기는 가장 긴 `env_release`입니다. 바이패스한 이펙트와 꺼진 악기 층은 뺍니다. 효과가 겹치는 정확한 합이 아니라 넉넉한 어림값입니다.
꼬리는 0.5초 단위로 올리고 30초에서 자릅니다	어림값이 모자라서 꼬리가 잘리는 쪽보다 조금 남는 쪽이 낫습니다. 30초는 창의 입력 한도와 같고, 되먹임 0.95에 4초 딜레이 같은 극단 값이 무한히 늘어나는 것을 막습니다.
파일명에서 `₩ / : * ? " < >` 와 제어 문자를 `_`로 바꿉니다	
파일명 시각은 지역 시간입니다	사용자가 보는 시계와 같아야 파일 목록에서 찾기 쉽습니다.

7. 시험

- `docs/handoff/s1-11b_tests/export_unit.mjs`의 계획 부분(37건)이 확인합니다. 곡 끝은 33마디이고 클립이 없으면 0입니다. 전체, 루프, 선택, 직접 지정(9마디 처음부터 25마디 끝, 같은 마디는 한 마디 길이)의 범위가 맞고, 빈 곡과 루프 없음과 선택 없음과 잘못된 범위는 각각 이유를 돌려줍니다. 120BPM에서 한 마디는 2초입니다. 꼬리는 리버브 decay 2에 pre_delay 0.1이면 2.5초, 바이패스하면 0, 딜레이는 시간 0.5초에 되먹임 0.5이면 5초대, 되먹임 0이면 1초, 악기 릴리스 1.2초는 1.5초이고 극단 값도 30초를 넘지 않습니다. 파일명은 금지 글자와 공백과 앞의 점을 정리하고, 한글은 그대로 두며, 시각은 `20261020_1405`와 `20270103_0407`처럼 0을 채웁니다. 시간 표시와 크기 표시도 확인합니다.
- `docs/handoff/s1-11b_tests/ui_export.mjs`는 실제 화면에서 범위 설명(전체 `bar 1 – 6`, 선택한 클립 `bar 5 – 6`, 직접 지정 `bar 3 – 4`)과 파일명 형식을 확인합니다.